Dependent Types: the System λP

Lecture 15

Section 1

Part I: Recall — The λ-Cube So Far

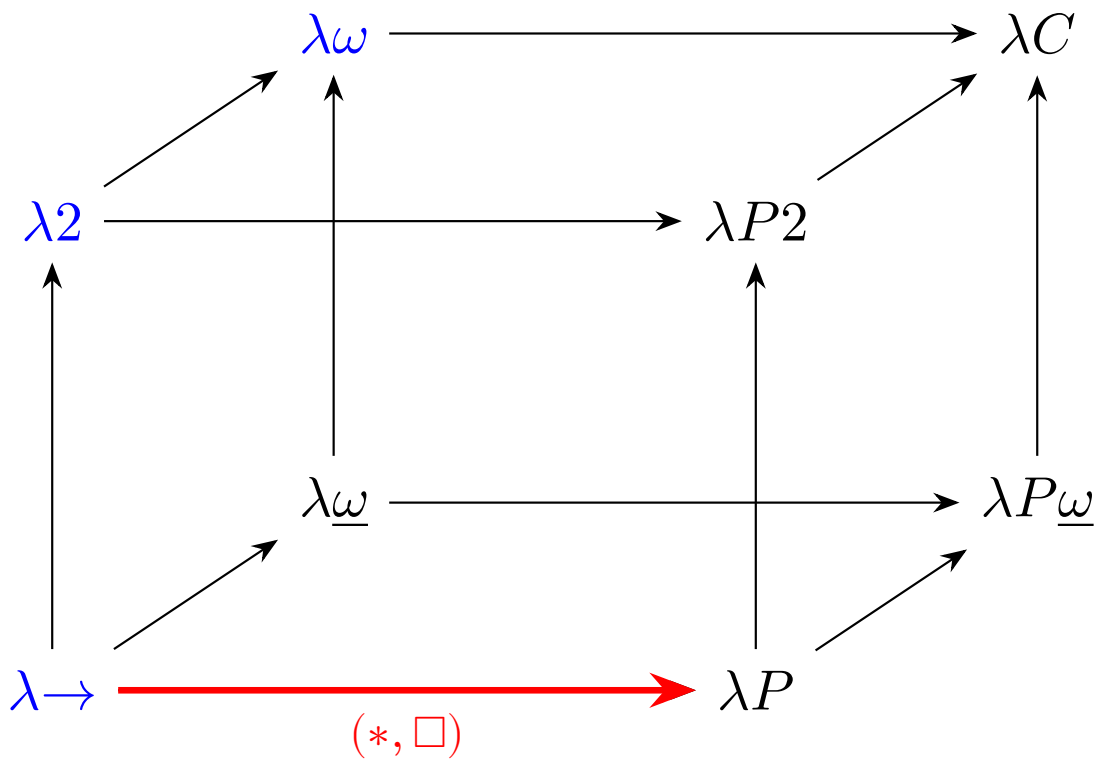
Where we are on the cube

Last time we introduced Barendregt's λ-cube and recovered our old friends:

Vertex	**Rules**	**Old name**
$\lambda{\to}$	$\{(*,*)\}$	STLC
$\lambda 2$	$\{(*,*),\ (\Box,*)\}$	System F
$\lambda\omega$	$\{(*,*),\ (\Box,*),\ (\Box,\Box)\}$	System Fω

All three sit on the **left face** of the cube. Today we move to the **right face** by adding the rule $(*,\Box)$.

The left face vs. the right face



Blue: systems we know. Red arrow: the step we take today. λP adds dependent types to STLC, **without** polymorphism or type operators.

Section 2

Part II: Why Types Should Depend on Terms

The limitation of System F

In System F, the type of a function says *nothing* about the relationship between input and output **values**:

$$\mathsf{head} : \forall \alpha.\, \mathsf{List}\ \alpha \to \alpha$$

This type does not tell us whether the list is empty. If we call head on an empty list, we get a runtime error.

The limitation of System F

In System F, the type of a function says *nothing* about the relationship between input and output **values**:

$$\mathsf{head} : \forall \alpha.\, \mathsf{List}\ \alpha \to \alpha$$

This type does not tell us whether the list is empty. If we call head on an empty list, we get a runtime error.

The problem: List α ignores the **length**. A list of 0 elements and a list of 1000 elements have the same type.

What we want to say

We want types that carry information about *values*:

- "a list of length n" $\longrightarrow$ Vec α n, where n : Nat is a **term**
- "an index in range $\{0, \ldots, n{-}1\}$" $\longrightarrow$ Fin n
- "an $m \times n$ matrix" $\longrightarrow$ Matrix $\mathbb{R}$ m n

In each case, a term appears *inside* the type. The type depends on a value.

A safe head function

With dependent types (working in a context $\alpha : *$), a safe head would have type:

$$\text{head} : \Pi n{:}\text{Nat}.\ \text{Vec}\ \alpha\ (\text{S}\ n) \to \alpha$$

The argument must be a vector of length S n (at least 1). An empty vector has type Vec α 0, which does **not** match Vec α (S n) for any n.

The type rules out the error statically.

{Note: the $\forall\alpha$ in the System F version would require the additional rule $(\Box, *)$, which λP does not have. In pure λP, we work at a fixed type α.}

The Curry–Howard motivation

Recall the Curry–Howard table:

STLC	propositional logic
System F	2nd-order propositional logic

What is conspicuously absent? **Predicate logic.** We cannot write "for all x, $P(x)$'' where x ranges over elements of some domain, because that requires a type $\Pi x{:}A.\, P(x)$ where $P(x)$ depends on the *term* x.

The Curry–Howard motivation

Recall the Curry–Howard table:

STLC	propositional logic
System F	2nd-order propositional logic

What is conspicuously absent? **Predicate logic.** We cannot write "for all x, $P(x)$'' where x ranges over elements of some domain, because that requires a type $\Pi x{:}A.\, P(x)$ where $P(x)$ depends on the *term* x.

The rule $(*, \Box)$ is exactly what we need.

Section 3

Part III: The System λP

Definition

λP uses the same syntax and inference rules as every cube system, with:

$$\mathcal{R}_{\lambda P} = \{(*, *),\ (*, \Box)\}$$

The rule $(*, *)$ gives ordinary function types. The rule $(*, \Box)$ is new:

$$\frac{\Gamma \vdash A : * \qquad \Gamma, x{:}A \vdash B : \Box}{\Gamma \vdash (\Pi x{:}A.\, B) : \Box} \quad (*, \Box)$$

Here A is a type and B is a kind. The variable x ranges over *terms*, and the kind B may mention x.

Note: λP has no $(\Box, *)$ (no polymorphism) and no $(\Box, \Box)$ (no type operators).

What the rule $(*, \Box)$ means

The rule allows **type constructors that take a term as argument**:

Nat $\to *$ is the kind $\Pi n{:}\text{Nat}.\, *$. A type constructor of this kind takes a natural number and returns a type.

Example: Vec $\alpha : \text{Nat} \to *$ maps each n to the type of α-vectors of length n.

In previous systems, type constructors could only take *types* as arguments $(* \to *)$. Now they can take *terms* (Nat $\to *$). This is the essential novelty.

Non-dependent vs. dependent Π

Non-dependent ($x \notin \mathsf{FV}(B)$):

$$\Pi x{:}A.\, B \;=\; A \to B$$

The output type is fixed.

Dependent ($x \in \mathsf{FV}(B)$):

$$\Pi x{:}A.\, B(x)$$

The output type *varies* with the input.

Example: $\Pi n{:}\mathsf{Nat}.\, \mathsf{Vec}\ \mathsf{Nat}\ n$ is the type of a function where calling it with 3 gives a $\mathsf{Vec}\ \mathsf{Nat}\ 3$, but calling it with 7 gives a $\mathsf{Vec}\ \mathsf{Nat}\ 7$.

The application rule: substitution in types

In the non-dependent case, application is: $f : A \to B$ and $a : A$ give $f\ a : B$.

With dependent types, the output type is *computed* from the argument:

$$\frac{\Gamma \vdash f : \Pi x{:}A.\, B(x) \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B\{x := a\}}$$

The substitution $B\{x := a\}$ computes the output type from the specific argument.

Example: if $f : \Pi n{:}\mathsf{Nat}.\, \mathsf{Vec}\ \mathsf{Nat}\ n$ and $a = 3$, then $f\ 3 : \mathsf{Vec}\ \mathsf{Nat}\ 3$.

Section 4

Part IV: Derivations in λP

Warmup: the identity function

In λP, ordinary non-dependent functions still work. From context $\alpha : *$:

1. $[] \vdash * : \Box$ (axiom)
2. $\alpha : * \vdash \alpha : *$ (start)
3. $\alpha : *, x : \alpha \vdash x : \alpha$ (start)
4. $\alpha : * \vdash (\alpha \to \alpha) : *$ (product, $(*, *)$)
5. $\alpha : * \vdash \lambda x{:}\alpha.\, x : \alpha \to \alpha$ (abstraction)

But we **cannot** close this with $\Pi\alpha{:}*\,.\,(\alpha \to \alpha)$ because λP lacks $(\Box, *)$. The identity works at a fixed type.

Deriving a dependent kind: $\alpha \to *$

Now we use the new rule. From context $\alpha : *$:

1. $[] \vdash * : \Box$ (axiom)
2. $\alpha : * \vdash \alpha : *$ (start)
3. $\alpha : *, x : \alpha \vdash * : \Box$ (weakening)
4. $\alpha : * \vdash (\Pi x{:}\alpha.\, *) : \Box$ **(product, $(*, \Box)$, using (2) and (3))**

So $\alpha \to * : \Box$. This is the kind of **predicates on** α: functions that take a term of type α and return a type ($=$ proposition).

Predicates as functions into $*$

A **predicate** on α is a function $p : \alpha \to *$ that assigns a proposition to each element.

Given $\alpha : *$, $p : \alpha \to *$, $x : \alpha$:

1. $p : \alpha \to *$ (start)
2. $x : \alpha$ (start)
3. $p\ x : *$ (application)

So $p\ x$ is a type — under Curry–Howard, a **proposition**. Intuitively: $p\ x$ is "p holds of x.''

A concrete example

Let Even : Nat $\to *$ be a predicate such that Even n is inhabited iff n is even.

Then:

- Even $4 : *$ is a type with a proof (it is inhabited)
- Even $3 : *$ is a type with no proof (it is empty)

A proof that all even numbers satisfy some property Q would have type:

$$\Pi n{:}\text{Nat}.\ \text{Even}\ n \to Q\ n$$

Read: for every n, if n is even, then $Q(n)$ holds.

Universal quantification

We can form the type $\Pi x{:}\alpha.\, p\ x$ (from context $\alpha : *,\ p : \alpha \to *$):

1. $\alpha : *,\ p : \alpha \to *,\ x : \alpha \vdash p\ x : *$ (as derived)
2. $\alpha : *,\ p : \alpha \to * \vdash (\Pi x{:}\alpha.\, p\ x) : *$ (product, $(*, *)$)

The type $\Pi x{:}\alpha.\, p\ x$ lives in sort $*$ — it is a proposition.

A term $f : \Pi x{:}\alpha.\, p\ x$ is a function that, given any $x : \alpha$, produces $f\ x : p\ x$ — a proof that p holds of x.

This is the universal quantifier: $\Pi x{:}\alpha.\, p\ x$ is $\forall x \in \alpha.\, p(x)$.

Proofs and instantiation

Introduction: given a proof term $e(x) : p\ x$ for each $x : \alpha$:

$$\lambda x{:}\alpha.\, e(x)\ :\ \Pi x{:}\alpha.\, p\ x$$

This is a proof of $\forall x.\, p(x)$.

Elimination (instantiation): given $f : \Pi x{:}\alpha.\, p\ x$ and $a : \alpha$:

$$f\ a\ :\ p\ a$$

From "for all x, $p(x)$'' and a specific a, deduce $p(a)$.

Introduction is λ-abstraction; elimination is application. The universal quantifier is **nothing but** the dependent function type.

Binary relations

A **binary relation** on α is $R : \alpha \to \alpha \to *$.

Reflexivity:

$$\Pi x{:}\alpha.\, R\ x\ x$$

Symmetry:

$$\Pi x{:}\alpha.\, \Pi y{:}\alpha.\, R\ x\ y \to R\ y\ x$$

Transitivity:

$$\Pi x{:}\alpha.\, \Pi y{:}\alpha.\, \Pi z{:}\alpha.\, R\ x\ y \to R\ y\ z \to R\ x\ z$$

These are exactly the standard definitions from first-order logic, written as types.

Implication between predicates

Given $\alpha : *$ and predicates $p, q : \alpha \to *$:

$$\Pi x{:}\alpha.\, p\ x \to q\ x$$

states: for every x, if $p(x)$ then $q(x)$.

We can also state: "pointwise implication lifts to universal closures'':

$$(\Pi x{:}\alpha.\, p\ x \to q\ x) \to (\Pi x{:}\alpha.\, p\ x) \to \Pi x{:}\alpha.\, q\ x$$

Implication between predicates

Given $\alpha : *$ and predicates $p, q : \alpha \to *$:

$$\Pi x{:}\alpha.\, p\ x \to q\ x$$

states: for every x, if $p(x)$ then $q(x)$.

We can also state: "pointwise implication lifts to universal closures'':

$$(\Pi x{:}\alpha.\, p\ x \to q\ x) \to (\Pi x{:}\alpha.\, p\ x) \to \Pi x{:}\alpha.\, q\ x$$

Proof: $\lambda h.\, \lambda f.\, \lambda x.\, h\ x\ (f\ x)$.

Section 5

Part V: Type Checking Requires Computation

The conversion rule in λP

Recall:

$$\frac{\Gamma \vdash t : A \qquad A =_\beta B \qquad \Gamma \vdash B : s}{\Gamma \vdash t : B}$$

In STLC and System F, types contain no terms, so $=_\beta$ between types is trivial.

In λP, types may contain terms. To check Vec Nat $(2 + 1) =_\beta$ Vec Nat 3, the type checker must **evaluate** $2 + 1$.

Type checking now involves computation.

Why strong normalization matters even more

If every well-typed term normalizes, then β-equivalence is decidable: reduce both sides to normal form and compare.

Without normalization, the type checker might loop: deciding Vec Nat $t_1 =$ Vec Nat t_2 requires evaluating t_1 and t_2.

Good news: λP, like all systems in the λ-cube, is strongly normalizing. Type checking is decidable.

And this is where the **parametric proof** pays off: strong normalization for the entire cube is proved once, covering λP as a special case.

Section 6

Part VI: The Logical Correspondence

λP is first-order predicate logic

λP	**Logic**
$A : *$	proposition
$t : A$	proof of A
$A \to B$	implication $A \Rightarrow B$
$\Pi x{:}\alpha.\, P(x)$	$\forall x \in \alpha.\, P(x)$
$\lambda x{:}\alpha.\, p$	proof of $\forall x.\, P(x)$
$f\ a$	instantiation: $P(a)$ from $\forall x.\, P(x)$

STLC handles propositional logic; System F handles 2nd-order propositional logic; λP handles **first-order predicate logic**.

Three systems, three logics

Cube vertex	**Logic**	**Can quantify over...**
$\lambda\!\rightarrow$ (STLC)	propositional	nothing
$\lambda 2$ (System F)	2nd-order propositional	types (= propositions)
λP	1st-order predicate	terms (= individuals)

$\lambda 2$ and λP are **independent**: they are reached from $\lambda\!\rightarrow$ along different axes. Combining them gives $\lambda P2$ (both quantifiers). Combining *all* axes gives λC (higher-order predicate logic).

Section 7

Part VII: Safe Programming and the Limits of λP

Safe vector operations

Working in a context $\alpha : *$, with Vec $\alpha\ n$ and Fin n:

Bounds-safe lookup:

$$\text{lookup} : \Pi n{:}\text{Nat}.\ \text{Vec}\ \alpha\ n \to \text{Fin}\ n \to \alpha$$

The index has type Fin n, so it is **guaranteed** in bounds.

Length-preserving map:

$$\text{map} : (\alpha \to \beta) \to \Pi n{:}\text{Nat}.\ \text{Vec}\ \alpha\ n \to \text{Vec}\ \beta\ n$$

The same n in input and output — length preservation is in the type.

More safe operations

Concatenation with precise length (in context $\alpha : *$):

$$\text{append} : \Pi m{:}\text{Nat}.\, \Pi n{:}\text{Nat}.\, \text{Vec}\ \alpha\ m \to \text{Vec}\ \alpha\ n \to \text{Vec}\ \alpha\ (m + n)$$

The output length is $m + n$ — verified statically.

Safe head:

$$\text{head} : \Pi n{:}\text{Nat}.\, \text{Vec}\ \alpha\ (\text{S}\ n) \to \alpha$$

Empty lists are ruled out: $\text{Vec}\ \alpha\ 0$ does not match $\text{Vec}\ \alpha\ (\text{S}\ n)$.

Errors that were runtime failures become **type errors**.

What λP cannot do

λP is the **simplest** dependent type system. It has serious limitations:

- **No polymorphism** ($(\Box, *)$ absent): we cannot write $\forall \alpha.\, \alpha \to \alpha$. Functions like map above work at *fixed* α, β.
- **No type operators** ($(\Box, \Box)$ absent): we cannot define List $: * \to *$ as a λ-expression.
- **First-order logic only**: we can quantify over elements, but not over types or predicates.

The path to λC

System	Has	Missing	Logic
λP	dep. types	poly., type ops.	1st-order pred.
$\lambda P2$	dep. types, poly.	type ops.	2nd-order pred.
$\lambda P\underline{\omega}$	dep. types, type ops.	poly.	1st-order + type ops.
λC	everything	nothing	higher-order pred.

λC — the **Calculus of Constructions** — sits at the apex. It is the core type theory behind Lean and Coq. That is the subject of the next lecture.

Section 8

Summary

What we covered

1. **The rule** $(*, \Box)$: types and kinds that depend on terms.
2. **The system** λP: $\mathcal{R} = \{(*, *), (*, \Box)\}$ — functions plus dependent types, but no polymorphism or type operators.
3. **Predicates and relations**: $p : \alpha \to *$ as predicates, $R : \alpha \to \alpha \to *$ as relations.
4. **Universal quantification**: $\Pi x{:}\alpha.\, p\ x$ is $\forall x.\, p(x)$; proofs are functions; instantiation is application.
5. **Type checking requires computation**: the conversion rule forces the type checker to normalize terms inside types.
6. **Limits**: no polymorphism, first-order logic only.

Next time

Next lecture: the Calculus of Constructions (λC).

λC combines all three axes: dependent types, polymorphism, and type operators. We will see polymorphic dependent functions, encode existential quantification, and reach higher-order predicate logic. We will also introduce **pure type systems**, which generalize the cube even further.